

Tutorial 03: Shader Fog - Teaching Guide

Overview

This tutorial introduces **uniform values** - a powerful way to pass data from your Python code to shaders. We'll use uniforms to implement a depth-based fog effect, learning how to perform calculations in the vertex shader along the way.

Prerequisites and Setup

Installation

Make sure you have completed the previous tutorials and have the virtual environment activated:

```
# Activate virtual environment (if not already active)
.\shader_tutorial_venv\Scripts\Activate.ps1

# Navigate to tutorials folder
cd tutorials

# Run tutorial 03
python 03_shader_fog.py
```

Expected Output: A window showing geometry that fades into white fog based on distance from the camera. Scaled 3× and rotated 45° for dramatic effect.

Interactive Controls:

- **Arrow Keys (UP/DOWN)** - Move camera forward/backward
- **Arrow Keys (LEFT/RIGHT)** - Move camera left/right
- **PAGE UP/DOWN** - Move camera up/down
- **ESC** - Exit

Why White Background is Critical:

The fog color is set to white (1, 1, 1, 1) and the background is also white. This creates a seamless fade where distant geometry gradually becomes indistinguishable from the background - just like real atmospheric fog! If the background were black, objects would appear as dark shapes on a contrasting background instead of naturally fading away.

Learning Objectives

By the end of this tutorial, students will understand:

1. What uniform values are and how they differ from varying values
2. How to pass data from CPU (Python) to GPU (shaders)

3. How to perform calculations in the vertex shader
4. How to create distance-based effects (fog)
5. Built-in GLSL functions (ftransform, clamp, mix, abs)
6. How to use transformation functions (glRotate, glScale)

What's New in This Tutorial

Compared to Tutorial 02, we're adding:

- ☒ **Uniform values** for CPU→GPU communication
- ☒ **Vertex shader calculations** (not just passing data through)
- ☒ **GLSL built-in functions** (ftransform, clamp, mix, abs)
- ☒ **Local variables** in shaders
- ☒ **Fog effects** based on depth
- ☒ **Scene transformation** (rotation, scaling)

Data Flow Comparison

Tutorial 01 (Basic):

```
Python → VBO → Vertex Shader → Fragment Shader → Screen
                (position)          (green color)
```

Tutorial 02 (Colors):

```
Python → VBO → Vertex Shader → Varying → Fragment Shader → Screen
                (pos + color)      (interpolated color)
```

Tutorial 03 (Fog with Uniforms):

```
Python → Uniforms —————
                (fog settings)
                        |
                        ↓
Python → VBO → Vertex Shader → Varying → Fragment Shader → Screen
                (pos + color)      (fog + distance)
```

Code Breakdown

1. Vertex Shader with Uniforms

```

uniform float end_fog;
uniform vec4 fog_color;

void main() {
    float fog;           // amount of fog to apply
    float fog_coord;     // distance for fog calculation

    gl_Position = ftransform();

    fog_coord = abs(gl_Position.z);
    fog_coord = clamp( fog_coord, 0.0, end_fog);

    fog = (end_fog - fog_coord)/end_fog;
    fog = clamp( fog, 0.0, 1.0);

    gl_FrontColor = mix(fog_color, gl_Color, fog);
}

```

Let's break this down piece by piece...

2. Understanding Uniforms

```

uniform float end_fog;
uniform vec4 fog_color;

```

What is a Uniform?

- A variable that is the **same** (uniform) for all vertices in a draw call
- Set once from Python, used by all vertices and fragments
- Read-only in shaders (you can't modify them)

Uniform vs Varying:

Feature	Uniform	Varying
Direction	CPU → GPU	Vertex shader → Fragment shader
Frequency	Once per draw call	Once per vertex
Interpolated?	No	Yes
Examples	Time, light position, fog settings	Color, texture coords, normals

Why use uniforms?

- Pass settings/parameters to shaders
- Control effects without recompiling shaders
- Can change every frame for animation

Visual:

Python Code:

```
glUniform1f(location, 15.0) ← Set once
```

↓

Vertex Shader (called for EVERY vertex):

```
uniform float end_fog; ← ALL vertices see 15.0
```

```
  |      |      |  
v1     v2     v3 ... (each uses same value)
```

3. Local Variables in Shaders

```
float fog;           // amount of fog to apply  
float fog_coord;     // distance for fog calculation
```

What are local variables?

- Temporary storage during shader execution
- Each instance of the shader gets its own copy
- Not shared between vertex/fragment shaders

When to use them:

- Breaking complex calculations into steps
- Making code more readable
- Avoiding recalculation of same value

Scope:

```
void main() {  
    float x = 5.0;    // Local to this main() function  
    {  
        float y = 3.0; // Local to this block  
    }  
    // y is not accessible here  
}
```

4. The ftransform() Function

```
gl_Position = ftransform();
```

What it does:

- Optimized version of: `gl_ModelViewProjectionMatrix * gl_Vertex`
- Guaranteed to produce identical results across runs
- Slightly faster on some hardware

Why use it?

1. **Consistency:** Important for multi-pass rendering
2. **Optimization:** GPU vendor can optimize it specially
3. **Best practice:** Recommended by OpenGL spec

When transformation happens:

```
Model Space (your VBO coordinates)
  ↓ Model Matrix
World Space (object in world)
  ↓ View Matrix
Eye/Camera Space (relative to camera)
  ↓ Projection Matrix
Clip Space (gl_Position)
  ↓ (automatic)
Screen Space (pixels)
```

`frtransform()` does Model → World → Eye → Clip in one go!

5. Calculating Distance (Z-coordinate)

```
fog_coord = abs(gl_Position.z);
```

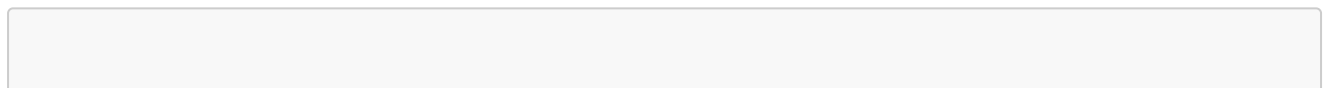
What is `gl_Position.z`?

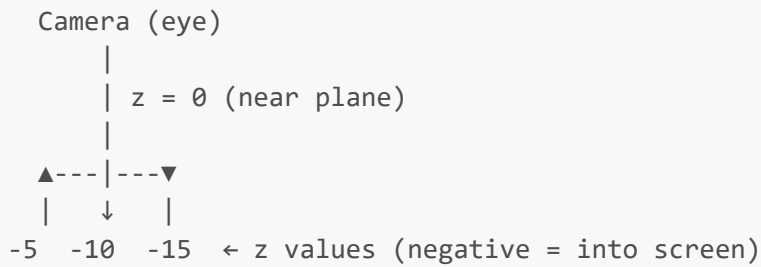
- After transformation, z represents "distance into screen"
- Negative z = in front of camera
- Positive z = behind camera (usually clipped)
- More negative = further away

Why `abs()`?

- Makes distance always positive
- Simplifies fog calculations
- Handles both directions uniformly

Visual:





abs(z): 5 10 15 ← distance we use for fog

6. Clamping Values

```
fog_coord = clamp( fog_coord, 0.0, end_fog);
```

What clamp() does:

```
clamp(value, min, max) returns:
- min if value < min
- max if value > max
- value otherwise
```

Examples:

```
clamp(5.0, 0.0, 10.0) → 5.0   (within range)
clamp(-2.0, 0.0, 10.0) → 0.0   (below minimum)
clamp(15.0, 0.0, 10.0) → 10.0   (above maximum)
```

Why clamp here?

- Prevents negative distances
- Prevents fog calculation from getting too large
- Ensures fog_coord is in range [0, end_fog]

7. Calculating Fog Factor

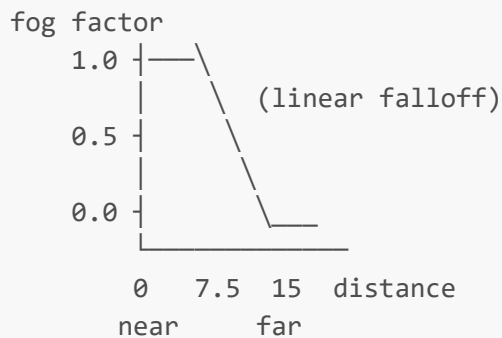
```
fog = (end_fog - fog_coord)/end_fog;
fog = clamp( fog, 0.0, 1.0);
```

The Math:

If `end_fog = 15`:

- At distance 0: $\text{fog} = (15 - 0)/15 = 1.0 \rightarrow$ **no fog** (100% visible)
- At distance 7.5: $\text{fog} = (15 - 7.5)/15 = 0.5 \rightarrow$ **half fog**
- At distance 15: $\text{fog} = (15 - 15)/15 = 0.0 \rightarrow$ **full fog** (0% visible)

Graph:



Why clamp again?

- Safety: ensure fog is always 0 to 1
- Handles edge cases
- Required by `mix()` function

8. Mixing Colors

```
gl_FrontColor = mix(fog_color, gl_Color, fog);
```

What `mix()` does:

```
mix(a, b, t) returns:  
a * (1-t) + b * t
```

With our fog factor:

- $\text{fog} = 1.0 \rightarrow \text{mix}(\text{fog_color}, \text{gl_Color}, 1.0) = \text{gl_Color}$ (no fog, original color)
- $\text{fog} = 0.5 \rightarrow$ 50% `fog_color` + 50% `gl_Color` (half fog)
- $\text{fog} = 0.0 \rightarrow \text{fog_color}$ (full fog, can't see object color)

Examples:

```
// fog_color = white (1,1,1,1), gl_Color = red (1,0,0,1)  
  
fog = 1.0: mix(white, red, 1.0) = red      ← near
```

```
fog = 0.5: mix(white, red, 0.5) = pink (1,0.5,0.5) ← medium  
fog = 0.0: mix(white, red, 0.0) = white ← far
```

Why `gl_FrontColor`?

- Built-in varying that automatically passes to fragment shader
- Shows up as `gl_Color` in fragment shader
- Legacy but convenient

9. Fragment Shader (Simple!)

```
void main() {  
    gl_FragColor = gl_Color;  
}
```

Why so simple?

- All the work was done in the vertex shader!
- Fog is already "baked into" the color
- We just use the interpolated color

Could we do fog in fragment shader?

Yes! But it's more expensive:

- Vertex shader: run 3 times per triangle (3 vertices)
- Fragment shader: run hundreds/thousands of times (many pixels)

Rule of thumb: Do as much as possible in vertex shader!

10. Getting Uniform Locations

```
self.UNIFORM_LOCATIONS = {  
    'end_fog': glGetUniformLocation( self.shader, 'end_fog' ),  
    'fog_color': glGetUniformLocation( self.shader, 'fog_color' ),  
}
```

What it does:

- Queries the compiled shader for uniform locations
- Returns an integer "handle" for each uniform
- Think of it like getting a "variable address"

Why query locations?

- Shader compiler may rearrange/optimize uniforms

- Location is not guaranteed to match declaration order
- Must query after compilation

Important: Query locations in `OnInit()`, use them in `Render()`

11. Setting Uniform Values

```
glUniform1f( self.UNIFORM_LOCATIONS['end_fog'], 15)
glUniform4f( self.UNIFORM_LOCATIONS['fog_color'], 1, 1, 1, 1)
```

glUniform Functions:

Function	Arguments	Shader Type
<code>glUniform1f(loc, x)</code>	1 float	uniform float
<code>glUniform2f(loc, x, y)</code>	2 floats	uniform vec2
<code>glUniform3f(loc, x, y, z)</code>	3 floats	uniform vec3
<code>glUniform4f(loc, x, y, z, w)</code>	4 floats	uniform vec4
<code>glUniform1i(loc, x)</code>	1 int	uniform int
<code>glUniformMatrix4fv(loc, ...)</code>	4×4 matrix	uniform mat4

Examples:

```
# Float
glUniform1f(loc, 3.14)

# Vector
glUniform3f(loc, 1.0, 0.5, 0.0)

# Color (RGBA)
glUniform4f(loc, 1.0, 1.0, 1.0, 1.0) # white

# From array
import numpy as np
color = np.array([1, 1, 1, 1], 'f')
glUniform4fv(loc, 1, color)
```

When to set uniforms:

- After `glUseProgram()` activates the shader
 - Before `glDrawArrays()` draws geometry
 - Can change every frame for animation!
-

12. Transformation Functions

```
glRotate( 45, 0, 1, 0 )  
glScale( 3, 3, 3 )
```

What they do:

- Modify the Model-View matrix
- Transform geometry before rendering
- Legacy functions (still work!)

glRotate(angle, x, y, z):

- **angle**: degrees to rotate
- **x, y, z**: axis of rotation
- Here: 45° around Y-axis (vertical)

glScale(x, y, z):

- **x, y, z**: scale factors for each axis
- Here: 3× bigger in all directions

Effect on our scene:

Original: After Transform:



(rotated 45° and scaled 3×)

Why transform?

- Makes fog effect more visible
- Geometry spans a wider distance range
- More dramatic depth cue

Order matters!

```
glRotate(45, 0, 1, 0) ← Applied SECOND (to rotated object)  
glScale(3, 3, 3)      ← Applied FIRST (to original object)
```

Order is reverse of how you write them!

Complete Data Flow

Let's trace one vertex through the entire pipeline:

Python Setup:

```
VBO vertex: [2, -1, 0, 1, 0, 0] (position=2,-1,0, color=red)
Uniforms: end_fog=15, fog_color=white(1,1,1,1)
Transform: rotate 45°, scale 3×
```

Vertex Shader:

```
1. gl_Vertex = (2, -1, 0)
2. gl_Color = (1, 0, 0) (red)
3. gl_Position = ftransform()
   After rotation+scale: ~ (6*cos(45°), -3, 6*sin(45°)) ≈ (4.24, -3, 4.24)
4. fog_coord = abs(4.24) = 4.24
5. fog_coord clamped to [0, 15] = 4.24
6. fog = (15 - 4.24)/15 = 0.717
7. fog clamped to [0, 1] = 0.717
8. gl_FrontColor = mix(white, red, 0.717)
   = (1,1,1)*(0.283) + (1,0,0)*(0.717)
   = (1, 0.283, 0.283) ← pinkish red
```

Rasterization:

- Creates fragments
- Interpolates colors across triangle

Fragment Shader:

```
gl_Color = (1, 0.283, 0.283) (interpolated)
gl_FragColor = (1, 0.283, 0.283)
```

Result: Red geometry with slight white fog tint!

Visual Explanation of Fog

Without Fog:

Camera Object
👁️ → 🟤 (pure red)

Color at any distance: `rgb(1, 0, 0)`

With Fog:

Camera



Distance = 0: fog = 1.0 → red (no fog)
Distance = 7: fog = 0.5 → pink (50% fog)
Distance = 15: fog = 0.0 → white (full fog)

Atmospheric Perspective:

This mimics reality! In the real world:

- Nearby objects: clear, saturated colors
- Medium distance: slightly hazy
- Far distance: fade into atmospheric color (usually blue/grey)

Artists use this for depth perception in 2D images!

Built-in GLSL Functions Used

1. **ftransform()**

```
vec4 ftransform()
```

- Returns: transformed vertex position
- Equivalent to: `gl_ModelViewProjectionMatrix * gl_Vertex`
- Optimized and consistent

2. **abs()**

```
float abs(float x)
```

- Returns: absolute value
- Examples: `abs(-5.0) = 5.0`, `abs(3.0) = 3.0`

3. **clamp()**

```
float clamp(float x, float min, float max)
```

- Returns: x constrained to [min, max]
- Examples:
 - `clamp(5, 0, 10) = 5`

- `clamp(15, 0, 10) = 10`
- `clamp(-5, 0, 10) = 0`

4. `mix()`

```
vec4 mix(vec4 a, vec4 b, float t)
```

- Returns: linear interpolation between a and b
- Formula: $a * (1-t) + b * t$
- Examples:
 - `mix(black, white, 0.0) = black`
 - `mix(black, white, 0.5) = grey`
 - `mix(black, white, 1.0) = white`

Other useful GLSL functions:

- `min(a, b), max(a, b)` - minimum/maximum
- `sqrt(x)` - square root
- `pow(x, y)` - power (x^y)
- `sin(x), cos(x), tan(x)` - trigonometry
- `length(v)` - vector length
- `normalize(v)` - unit vector
- `dot(a, b)` - dot product
- `cross(a, b)` - cross product

Common Student Questions

Q: Why calculate fog in vertex shader instead of fragment shader?

A: Performance! Vertex shader runs once per vertex (9 times), fragment shader runs once per pixel (thousands of times). But if you need per-pixel accuracy, fragment shader is better.

Q: What happens to objects beyond end_fog distance?

A: They're completely replaced by fog_color (you can't see them anymore).

Q: Can I make the fog non-linear (exponential)?

A: Yes! Replace the linear calculation:

```
// Exponential fog
fog = exp(-fog_coord * density);
```

Q: Why is fog_color white?

A: To match the background! If background was black, fog_color should be black, otherwise objects "glow" as they fade.

Q: Can I animate the fog?

A: Yes! Change uniforms every frame:

```
def Render(self):
    time = self.getTime()
    fog_distance = 10 + 5 * sin(time) # Pulsing fog
    glUniform1f(self.UNIFORM_LOCATIONS['end_fog'], fog_distance)
```

Q: What if I don't want fog at all for some objects?

A: Set `fog = 1.0` (or use a different shader without fog).

Q: Can I have colored fog (like green toxic fog)?

A: Yes! Set `fog_color` to green:

```
glUniform4f(self.UNIFORM_LOCATIONS['fog_color'], 0, 1, 0, 1) # green
```

Q: What's the difference between `gl_Color` and `gl_FrontColor`?

A:

- In vertex shader: write to `gl_FrontColor`
- In fragment shader: read from `gl_Color`
- They're connected (`gl_FrontColor` → `gl_Color`)

Experiments to Try

1. Change Fog Distance

```
glUniform1f(self.UNIFORM_LOCATIONS['end_fog'], 5) # Very close
glUniform1f(self.UNIFORM_LOCATIONS['end_fog'], 30) # Very far
```

Effect: Changes how quickly geometry fades

2. Colored Fog

```
# Blue fog (like atmosphere)
glUniform4f(self.UNIFORM_LOCATIONS['fog_color'], 0.5, 0.7, 1.0, 1)

# Green toxic fog
glUniform4f(self.UNIFORM_LOCATIONS['fog_color'], 0, 0.8, 0, 1)

# Red danger fog
glUniform4f(self.UNIFORM_LOCATIONS['fog_color'], 1, 0, 0, 1)
```

3. Exponential Fog

```
// In vertex shader, replace linear fog calculation:
fog_coord = abs(gl_Position.z);
fog = exp(-fog_coord * 0.2); // 0.2 is density
fog = clamp(fog, 0.0, 1.0);
```

Effect: More realistic, gradual fade

4. Squared Fog (Exponential²)

```
fog_coord = abs(gl_Position.z);
fog = exp(-fog_coord * fog_coord * 0.01);
```

Effect: Very clear near, rapid falloff

5. Animate Fog

```
def Render(self, mode=0):
    # ... setup ...

    import math
    import time
    fog_dist = 10 + 5 * math.sin(time.time())
    glUniform1f(self.UNIFORM_LOCATIONS['end_fog'], fog_dist)
```

Effect: Pulsing fog effect

6. Height-Based Fog

```
// In vertex shader
fog_coord = abs(gl_Position.y); // Use Y instead of Z
// Now fog is based on height, not distance!
```

7. Fog Only Far Away (Start Distance)

```
// Add start_fog uniform
uniform float start_fog;
uniform float end_fog;

// In main():
```

```
fog_coord = clamp(fog_coord, start_fog, end_fog);  
fog = (end_fog - fog_coord) / (end_fog - start_fog);
```

Debugging Tips

Problem: No fog effect visible

- Check uniform values are being set
- Verify end_fog is reasonable (try small value like 5)
- Make sure fog_color contrasts with geometry color
- Check that shader is being used

Problem: Everything is fog_color

- end_fog might be too small
- Check fog calculation logic
- Verify clamping isn't forcing fog=0

Problem: Compilation errors

- Uniform types must match usage
- Check for typos in uniform names
- Missing semicolons in shader code

Problem: Fog looks wrong

- Print fog values to debug:

```
gl_FrontColor = vec4(fog, fog, fog, 1); // Visualize fog factor
```

Problem: Fog doesn't match background

- Make sure fog_color matches clear color
- Check alpha channel (should be 1.0)

Performance Considerations

Vertex vs Fragment Fog:

Aspect	Vertex Fog	Fragment Fog
Performance	Fast (few vertices)	Slow (many fragments)
Quality	Per-vertex (interpolated)	Per-pixel (accurate)
Use case	Most situations	High quality needed

Optimization Tips:

1. Calculate fog in vertex shader when possible
2. Use built-in functions (they're optimized)
3. Minimize branching (if/else) in shaders
4. Keep uniforms to reasonable numbers (hardware limits exist)

Uniform Limits:

- Maximum ~1024-4096 uniforms per shader (varies by hardware)
 - Check with: `glGetIntegerv(GL_MAX_VERTEX_UNIFORM_COMPONENTS)`
-

Real-World Applications

Fog Effects Used In:

1. Games:

- Atmospheric depth
- Hide draw distance (pop-in)
- Create mood (horror games use fog heavily)
- Performance optimization

2. Simulation:

- Flight simulators (atmospheric haze)
- Driving simulators (weather effects)
- Scientific visualization

3. Film/CGI:

- Depth cues in 3D renders
- Volumetric lighting
- Atmospheric perspective

4. Other Techniques Using Similar Math:

- Depth of field blur
 - Ambient occlusion
 - Screen-space effects
-

Advanced Topics (For Later)

Volumetric Fog:

- Ray marching through 3D fog volume
- Samples density at multiple points
- Can have varying density

Fog with Transparency:

- Objects with alpha need special handling
- Fog applied before or after alpha blend?
- Order-dependent transparency issues

Multiple Fog Layers:

- Different fog colors at different heights
- Layered atmosphere effect
- Blending multiple fog calculations

Fog and Shadows:

- Shadowed regions might have different fog
- Light scattering through fog (god rays)
- Volumetric shadow mapping

Next Steps

You now know:

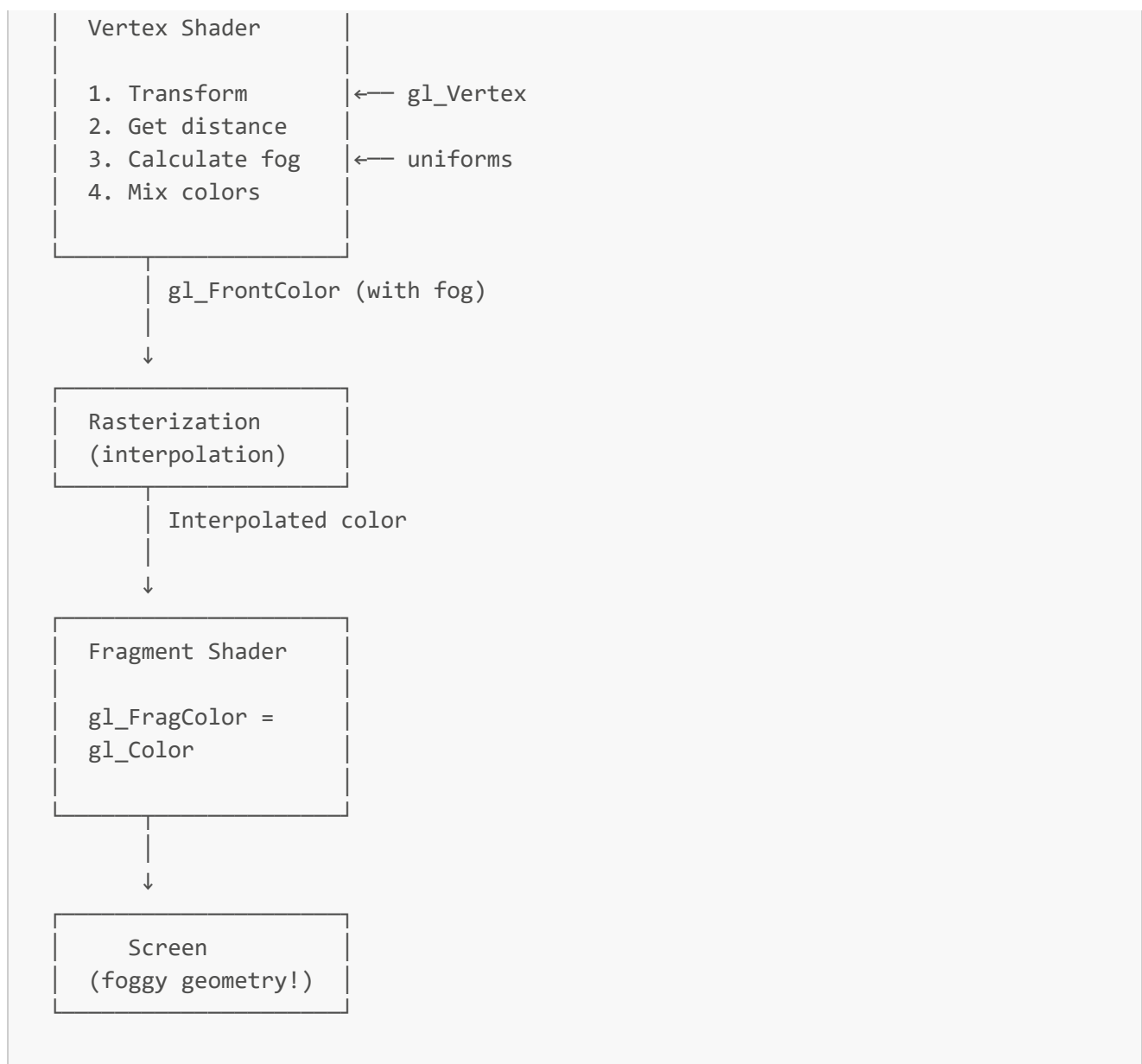
- ☒ Uniforms (CPU → GPU communication)
- ☒ Vertex shader calculations
- ☒ GLSL built-in functions
- ☒ Distance-based effects

Future tutorials might cover:

- Textures (images on geometry)
- Lighting calculations (normals, light sources)
- Advanced uniforms (matrices, arrays)
- Geometry shaders
- Compute shaders
- Modern GLSL (version 330+)

Summary Diagram





Congratulations!

You've now mastered:

1. **Tutorial 01:** Basic shaders and VBOs
2. **Tutorial 02:** Varying values and interpolation
3. **Tutorial 03:** Uniform values and shader calculations

You have the foundation to create:

- Custom visual effects
- Lighting systems
- Material systems
- Post-processing effects
- And much more!

The three key data paths:

- **Attributes** → vertex shader (position, color, etc.)
- **Uniforms** → both shaders (settings, transforms, etc.)
- **Varyings** → vertex → fragment (interpolated values)

Master these, and you master shaders! 🐝